Comfort: patient health records, connected

Hack Apertus Track 2B - Own Project | Team Comfort (solo) | 16 October 2026

Repository: github.com/sambutracy/Hospital-Records

Problem and solution

People receiving care across providers may have to reconstruct their own history from scattered letters, results and medication records. Comfort gives a patient a private place to preserve those originals, find relevant evidence and prepare a visit around the patient's stated concern. The patient selects sources and optional personal context, can inspect each original, and controls any sharing. The prototype organizes information; it does not diagnose, recommend treatment or replace a clinician's review.

Architecture and deployment

The application uses Django 5.2, a React/TypeScript client, PostgreSQL 18, Keycloak OIDC, Docker Compose and a serial asynchronous worker. PostgreSQL constraints protect ownership, immutable source versions, task snapshots and recipient-scoped temporary grants. Original files live in a private volume; derived suggestions do not modify them. Patient review is separate from saving a Journal entry, approving a packet or sharing records.

The demonstrated target is **on-premise**: the app, worker, database, identity service and model runner are administered together. Local inference runs on an internal Docker network; the model port is unpublished, weights are mounted read-only, and the runner has a read-only filesystem, dropped capabilities, a non-root user and resource limits. This is a development deployment, not an air-gap certification, Swiss-cloud deployment or production security assessment. Local HTTP, Keycloak development mode, TLS, operational backup/deletion and key-management procedures need production hardening.

Apertus and purposeful AI

Comfort uses **Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF**, a community Q4_K_M text-only conversion of `swiss-ai/Apertus-v1.5-8B` via `andreasmartin/apertus-v1.5-8b-text`; it is not the official multimodal checkpoint. The weight file is pinned to repository revision `248ec68a63e219e3f061e4c946fc8a20d63b9975` and SHA-256 `a037df8d87ff6caacee794ee85f55342f2152e0d359b7389033300c3bee5f299`. Inference uses the amd64 llama.cpp `server-b11429` image pinned by digest, 16,384-token context, six CPU threads, one generation slot, disabled thinking and bounded task-specific output. First setup downloads about 5.1 GB of weights; a local cache is verified and reused. There is no hosted API dependency in the default run path.

For related-record search and packet overviews, deterministic ranking narrows the authorized source set and Apertus selects source evidence. The server maps selected evidence back to immutable passages, checks exact quotations and retains links to originals. This is useful for locating and arranging evidence, but a valid quotation does not prove that a paraphrase is correct or that the source set is complete. Earlier local authenticated search and overview requests completed. Recorded development examples include a search processed in 7.11 seconds and an overview in 10.00 seconds; a different six-record overview took 54.73 seconds and cited two of six records. These are individual warm/local CPU runs, not a latency guarantee or accuracy estimate.

The newer structured visit brief asks Apertus to select and group up to 28 source-grounded items around visit purpose, medication/allergy context, history, symptoms, findings, documented follow-ups, function, care coordination and patient priorities. The model returns candidate identifiers, and the server displays the saved source statements and provenance. Patient-selected profile, medication, Journal and care context is opt-in and permission-checked; omitted sections remain unknown. Four deterministic development cases retained authored fact/section targets with no duplicate passages; 40 affected backend tests and 16 focused frontend tests passed, and the recorded broader Visit Brief frontend suite had 53 passing tests. **No live Apertus visit-brief result has yet been verified.**

Journal extraction, Health history AI and follow-up extraction are experimental. The deterministic Journal baseline on the frozen 23-case fictional corpus achieved 0.8377 field accuracy and 0.5909 exact-case accuracy across 22 non-empty cases; these are not Apertus scores. Real Journal `dev-03` attempts returned responses but failed strict validation, with missing/unsupported evidence among the observed failure stages. The later extraction correction still needs a live verification. Earlier AI features must not be represented as consistently reliable.

Evaluation and evidence

Evaluation	Result and scope
Related-record retrieval baseline	Literal keyword recall@5 0.5588 over 17 positive queries; correct abstention 3/3 on no-match cases. This is a deterministic baseline, not model performance.
Journal normalization baseline	0.8377 field accuracy and 0.5909 exact-case accuracy over 22 non-empty authored cases; not live Apertus.
Visit brief development screen	Four authored synthetic cases; deterministic planner retained targets with zero duplicate passages. No live model accuracy measured.
Recorded Visit Brief software checks	40 affected backend tests, 16 focused frontend tests; full related frontend run 53 tests. TypeScript, lint and production build passed in the recorded run.
Recorded interface checks	234 identity/records PostgreSQL tests and 61 frontend tests passed in their respective earlier runs; desktop and mobile browser checks were recorded separately. Counts are not combined into one suite.
Local Apertus examples	Real source-linked search and overview generations completed, with individual processing times above. No representative accuracy study or clinician review.

Other model evaluation attempts include invalid outputs and failed requests; no complete 23-case live Journal score or complete live Apertus retrieval/overview benchmark is claimed. The separate deterministic retrieval baseline was completed as reported above. The synthetic evaluations are authored engineering screens, not representative clinical data or a clinician-reviewed gold standard. Software tests use mocked inference where stated. No clinical outcome, medical correctness, accessibility conformance or clinician agreement was measured.

Safety, privacy and limits

Originals and derived text are distinct. Source citations must match preserved text; model suggestions require patient review. Optional health context defaults off and is frozen with its request; access is checked before inference and result reads. No model result changes a prescription, diagnosis, Journal history, sharing permission or clinical record. Family history retains attribution; a plan is not treated as a completed procedure; prescribed medication is not treated as confirmed use. No automatic sharing or diagnosis is performed.

Retrieval is bounded and may omit relevant records or qualifications. OCR for scanned documents and diagnostic image interpretation are not implemented. JPEG/PNG previews and bounded DICOM header inspection do not interpret MRI or ultrasound findings. Structured visit briefs are not live-validated. Local quantized CPU generation is asynchronous, single-slot and can be slow under load; first-time setup requires network access and substantial disk/RAM. The synthetic data exercises software contracts but does not establish safe clinical use. Professional identity, verified guardian authority, delegated assistance and voice controls are not verified features.

Reproducibility, licensing and references

From a clean repository checkout on an x86-64 host with Docker Linux containers, Compose v2 with `--wait`, GNU Make, about 16 GB host memory, 12 GB Docker allocation and 12 GB free disk, run `make run` at the repository root. Initial setup downloads containers, dependencies and pinned weights; the bootstrap creates private synthetic credentials. Open `http://localhost:8000/app/`. Do not commit `.env`, `.runtime/`, credentials, private records or model weights. Existing Docker volumes are retained. The detailed commands and demo limits are in `docs/demo-runbook.md`.

Code is Apache-2.0. Comfort-authored evaluation content is CDLA-Permissive-2.0. Imported synthetic documents retain their source licenses and attribution (including MIT and CC BY 4.0); this mixed collection is not relicensed as a whole. Hugging Face publication is optional and has not been performed. This report is released under CC BY 4.0.

References: Apertus v1.5 8B; pinned text conversion; llama.cpp Docker documentation; Hack Apertus Track 2B; organizer terms.